Multiprocessors and Caching

CS 2410

Wonsun Ahn

University of Pittsburgh

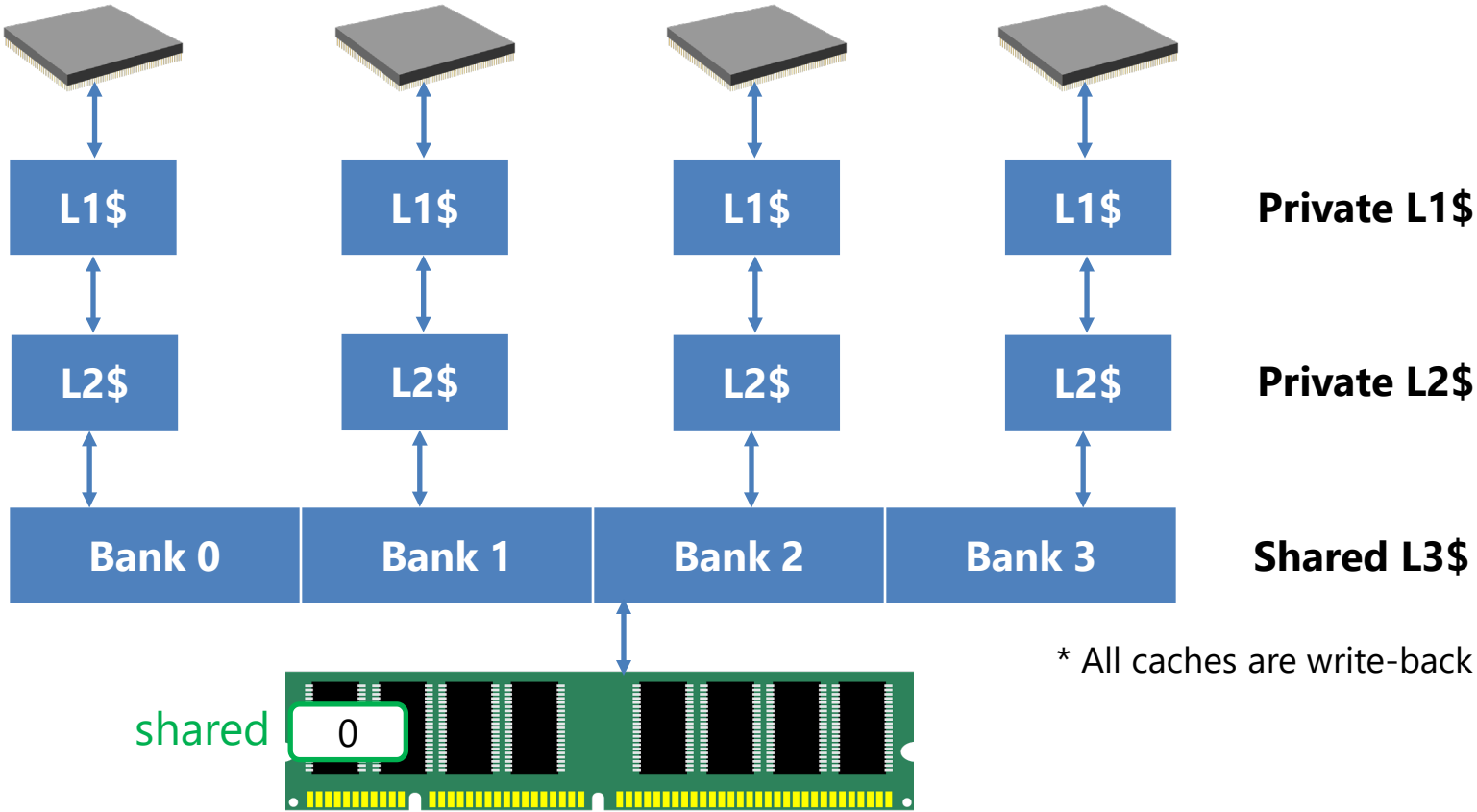
Two ways to use multiple processors

- **Distributed (Memory) System**
  - Processors **do not share memory** (and by extension **data**)
  - Processors exchange data through network messages
  - Programming standards:
    - Message Passing Interface (MPI) – C/C++ API for exchanging messages
    - Ajax (Asynchronous JavaScript and XML) – API for web apps
  - Data exchange protocols: TCP/IP, UDP/IP, JSON, XML…

- **Shared Memory System** (a.k.a. Multiprocessor System)
  - Processors **share memory** (and by extension **data**)
  - Programming standards:
    - Pthreads (POSIX threads), Java threads – APIs for threading
    - OpenMP – Compiler #pragma directives for parallelization
  - **Cache coherence protocol**: protocol for exchanging data among caches

Example of Cache Incoherence

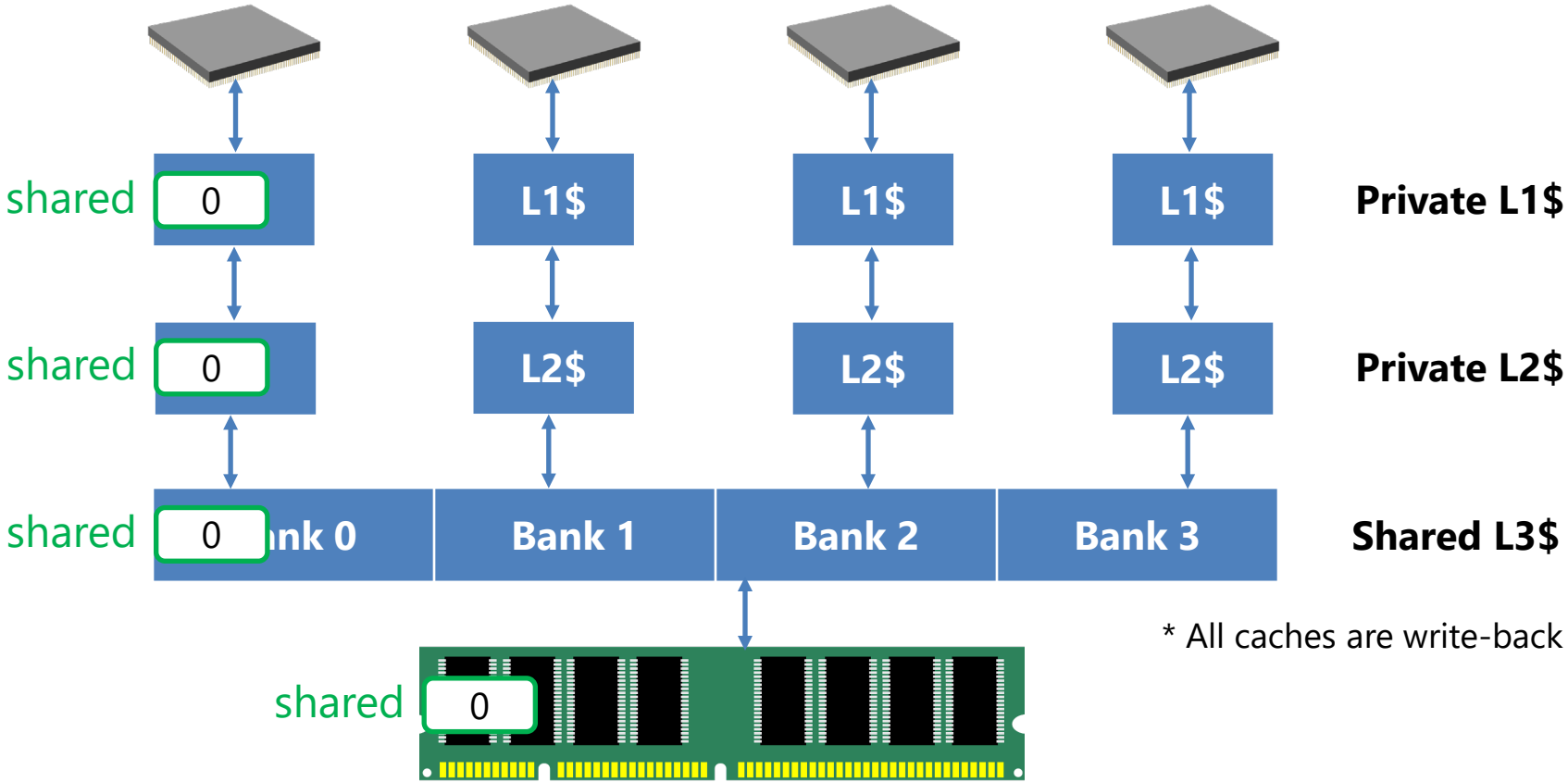
- What happens if caches sit in between processors and memory?



* All caches are write-back

Example of Cache Incoherence

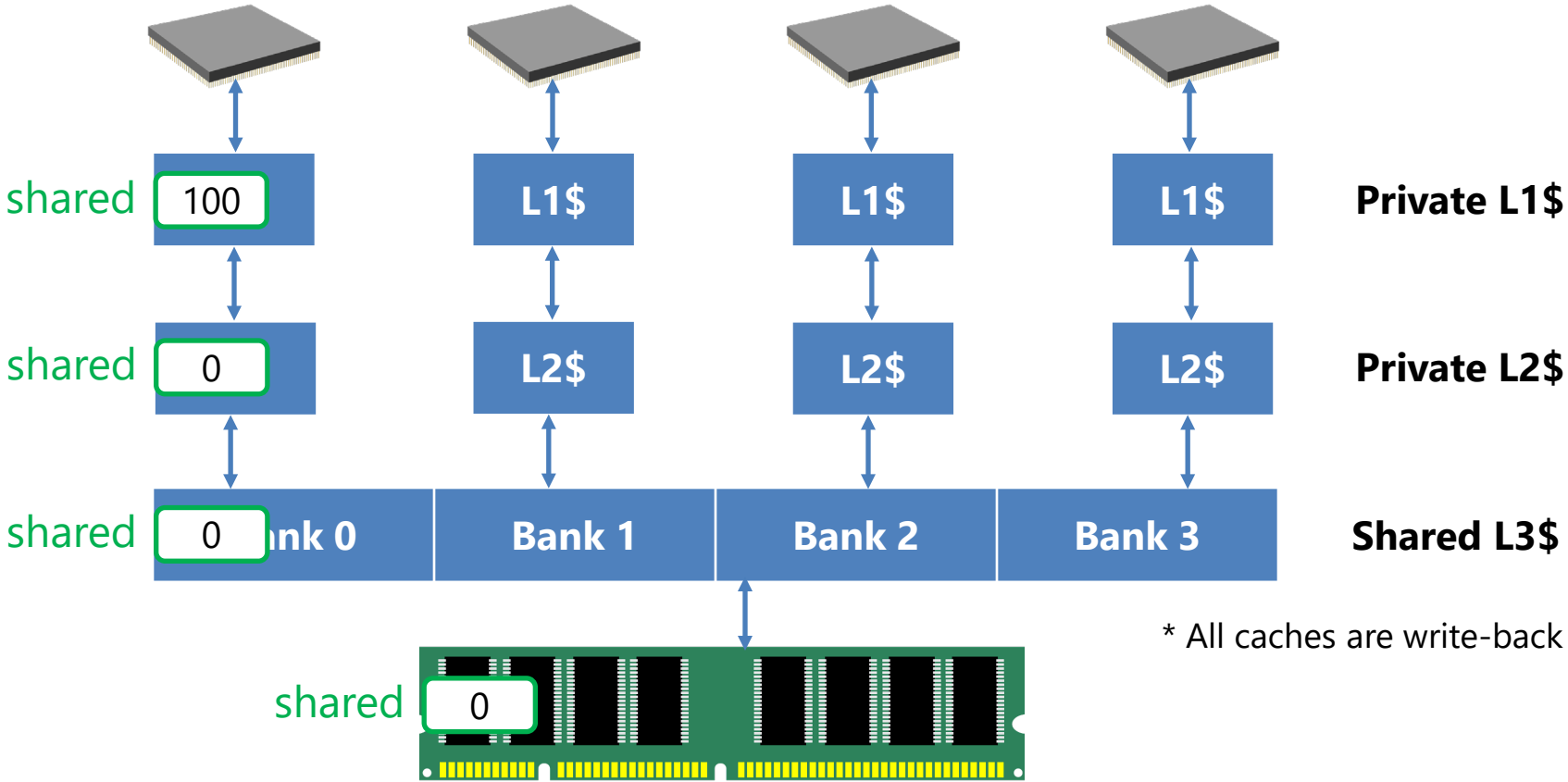
- Let's say CPU 0 first fetches variable `shared` for incrementing



* All caches are write-back

Example of Cache Incoherence

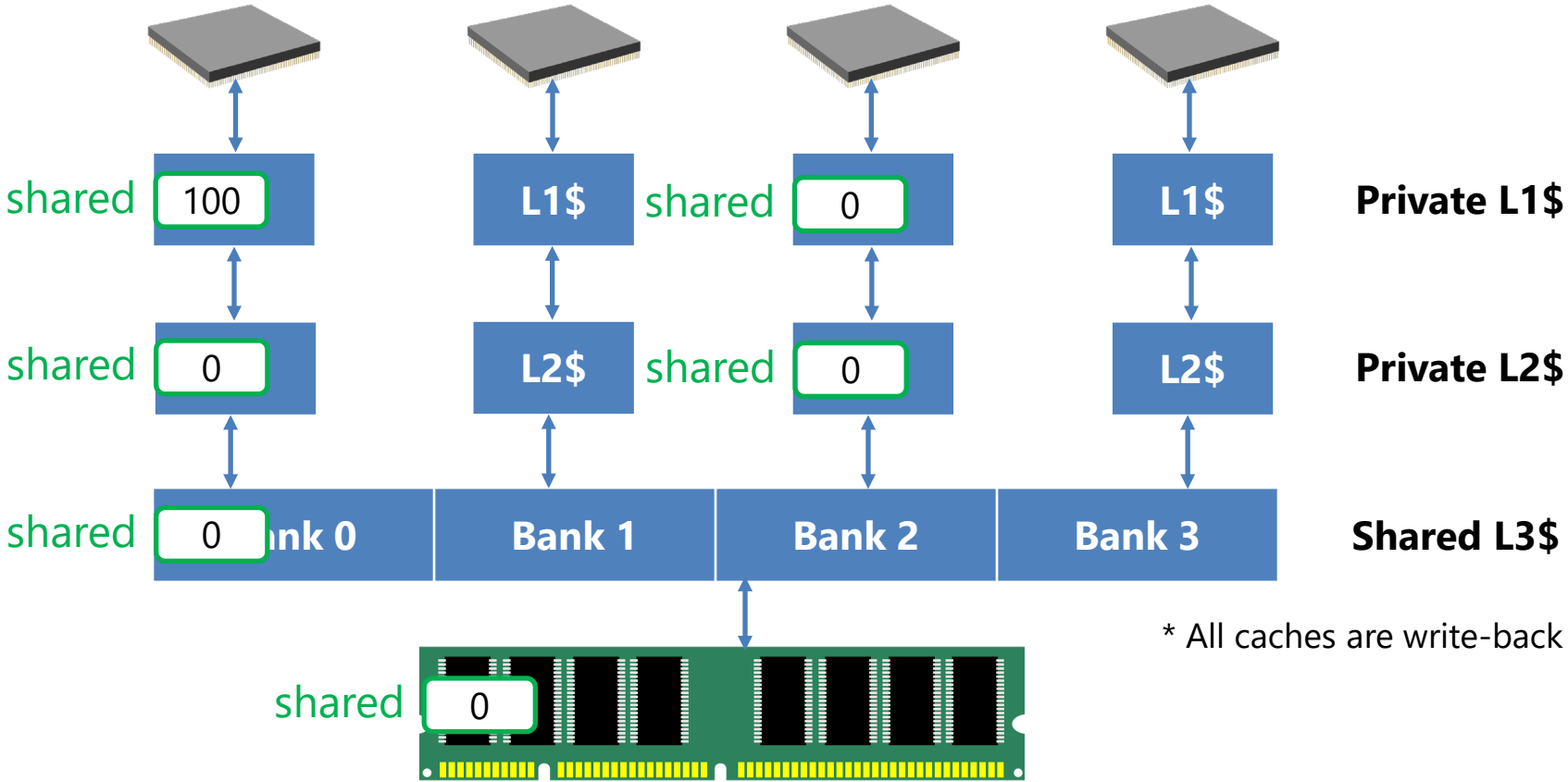
- Then CPU 0 increments `shared` 100 times to 100



* All caches are write-back

Example of Cache Incoherence

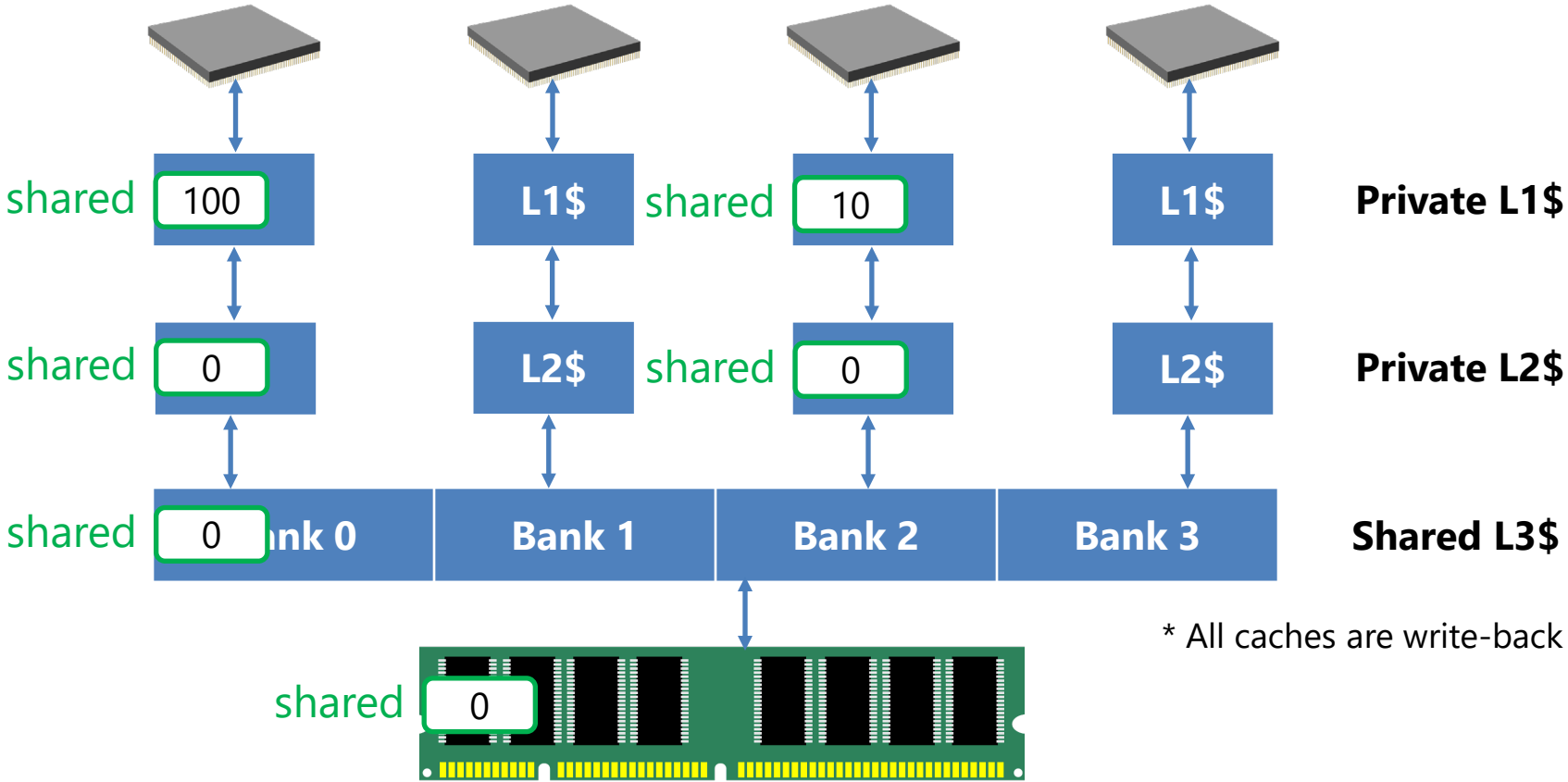
- Then CPU 2 gets hold of the mutex and fetches `shared` from L3



* All caches are write-back

Example of Cache Incoherence

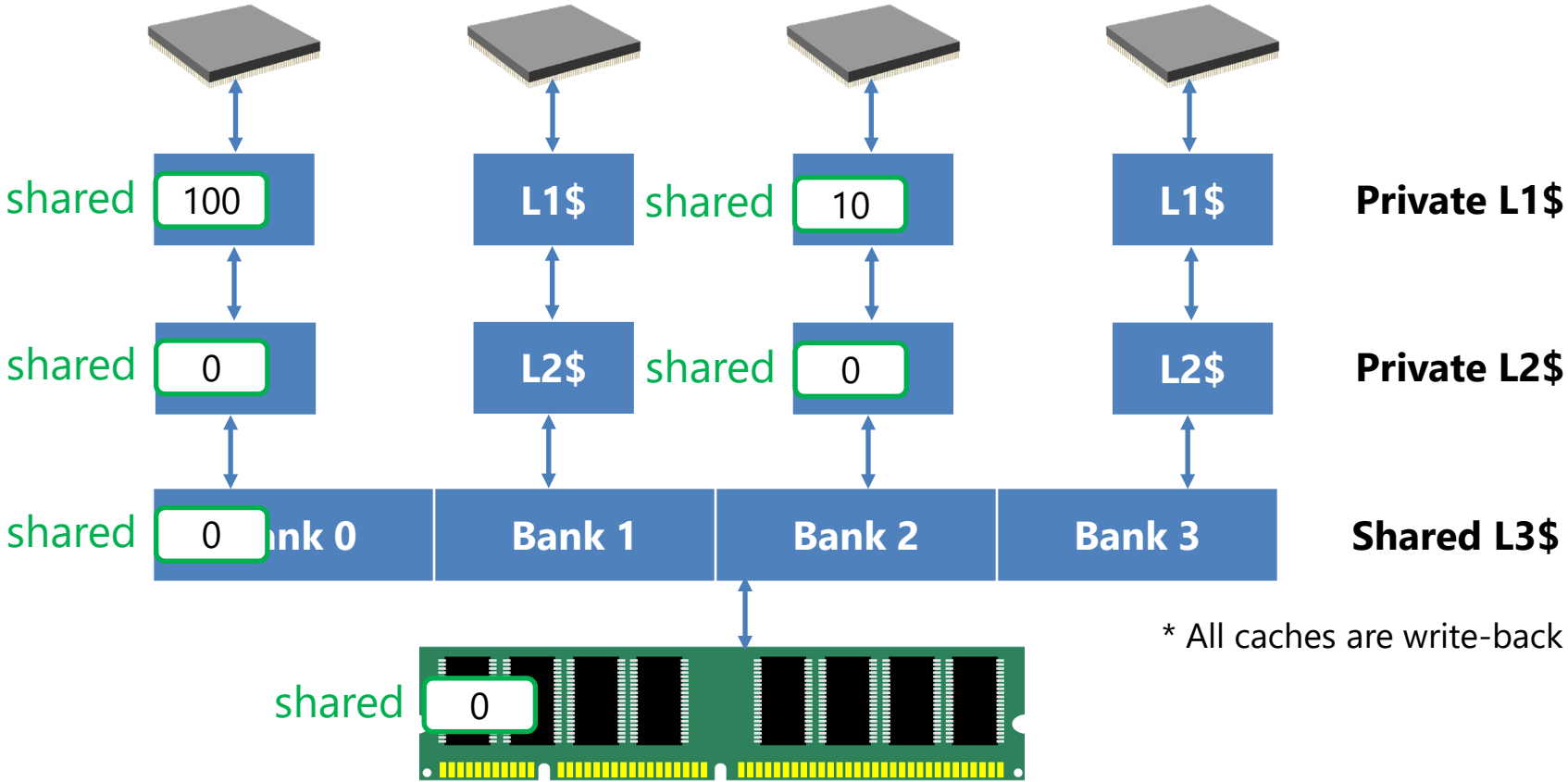
- Then CPU 2 increments `shared` 10 times to 10



* All caches are write-back

Example of Cache Incoherence

- Clearly this is wrong. L1 caches of CPU 0 and CPU 2 are **incoherent**.



* All caches are write-back

Cache Incoherence: Problem with Private Caches

- This problem does not occur with a **shared cache**.
  - All processors share and work on a **single copy** of data.

- The problem exists for **private** caches.
  - Private copy is at times inconsistent with lower memory.
  - **Incoherence** occurs when private copies differ from each other.
    → Means processors return different values for same location!

Cache Coherence

Cache Coherence

- **Cache coherence**:
  - All processors of system see the **same view of memory**
  - All processors see the **same value** for the **same address**

- With coherence, complexity of cache hierarchy is hidden from SW
  - To software, it appears to directly access value stored in memory (Except its faster, but performance is not part of the ISA)

Implementing Cache Coherence

- How to guarantee changes in value are propagated to all caches?

- **Cache coherence protocol**: A protocol, or set of rules, that all caches must follow to ensure coherence between caches
  - MSI (Modified-Shared-Invalid)
  - MESI (Modified-Exclusive-Shared-Invalid)
  - … often named after the **states** in cache controller **FSM**

MSI Cache Coherence Protocol

Implementing Cache Coherence

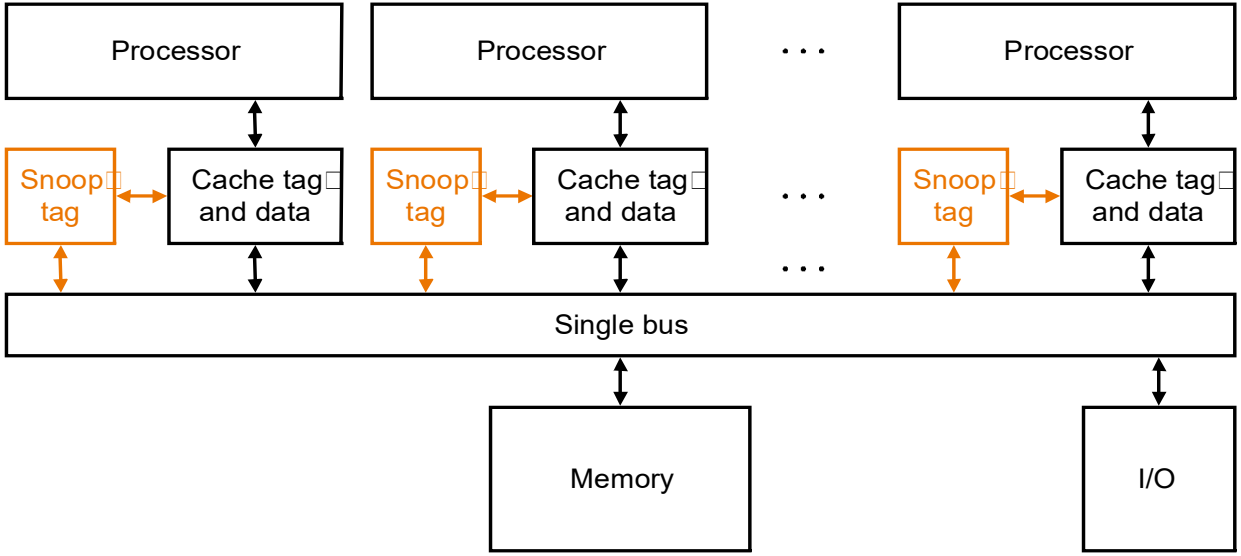
- How to guarantee changes in value are propagated to all caches?

- **Cache coherence protocol**: A protocol, or set of rules, that all caches must follow to ensure coherence between caches
  - MSI (Modified-Shared-Invalid)
  - MESI (Modified-Exclusive-Shared-Invalid)
  - … often named after the **states** in cache controller **FSM**

- Three states of **MSI** protocol (maintained for each cache block):
  - Modified: Dirty. Only this cache has copy.
  - Shared: Clean. Other caches may have copy.
  - Invalid: Block contains no data.

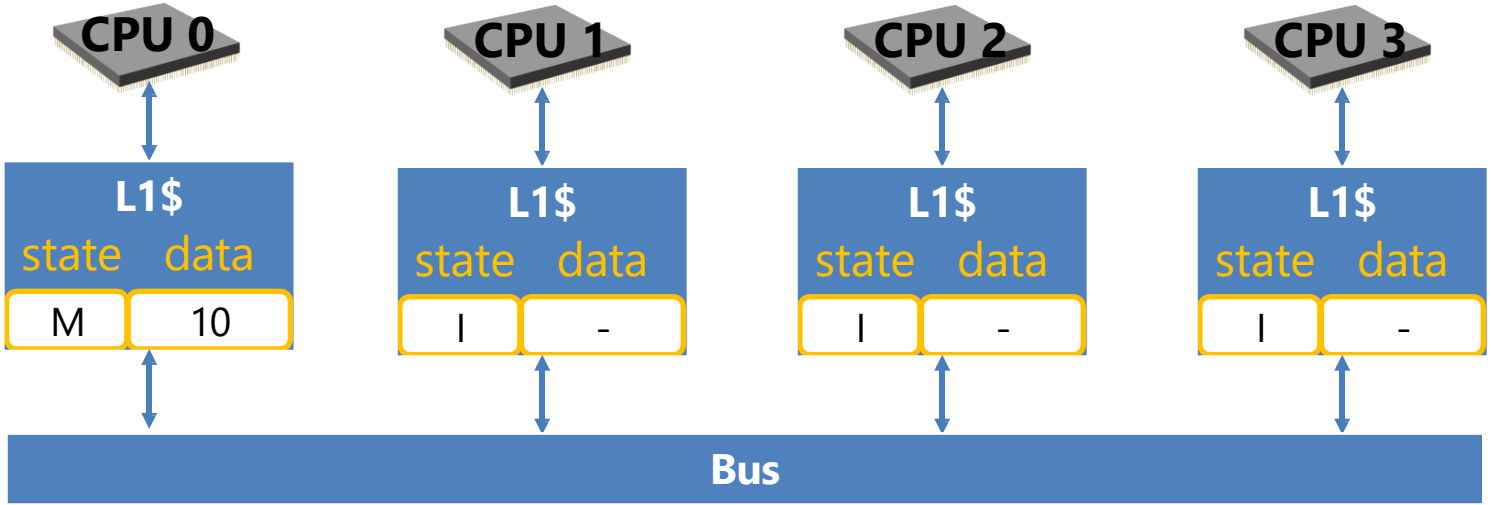
  ⇨ Coherence is maintained since two modified copies can't co-exist

MSI Snoopy Cache Coherence Protocol



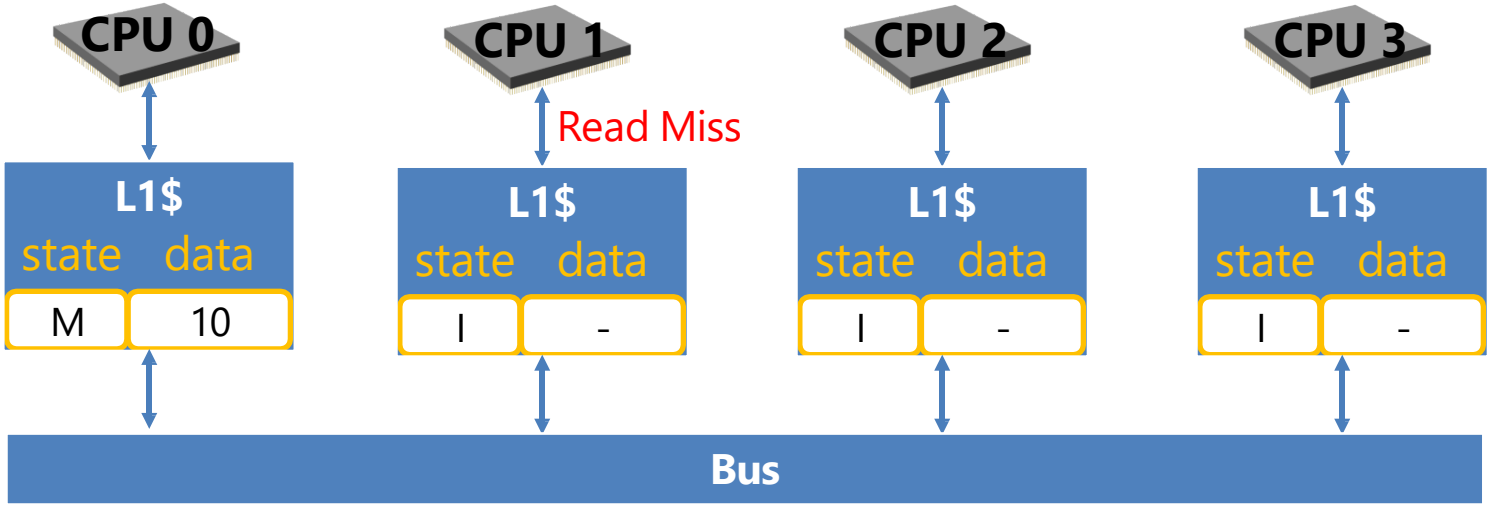
- Each processor **monitors (snoops)** the activity on the **bus**
  - In much the same way as how nodes snoop the Ethernet
- Cache state changes in response to both:
  - Read / writes from the **local processor**
  - Read misses / write misses from **remote processors** it snoops

Scenario 1: Read Snooped on Bus



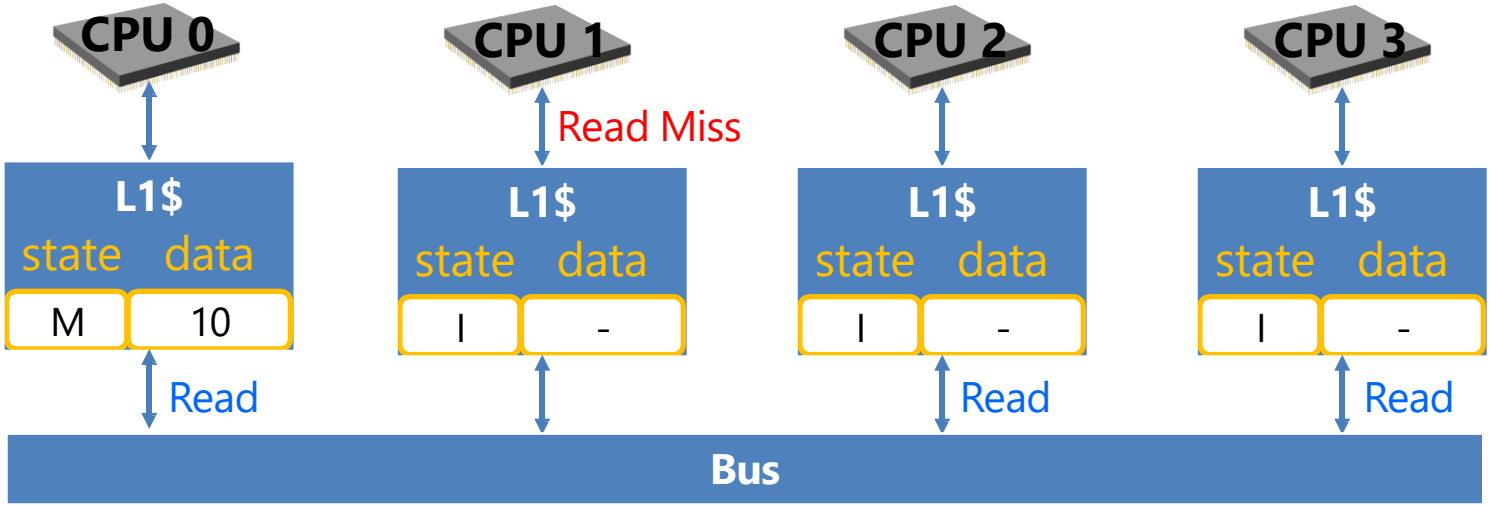
1. CPU 0: line in Modified state, CPU 1~3: line in Invalid state

Scenario 1: Read Snooped on Bus



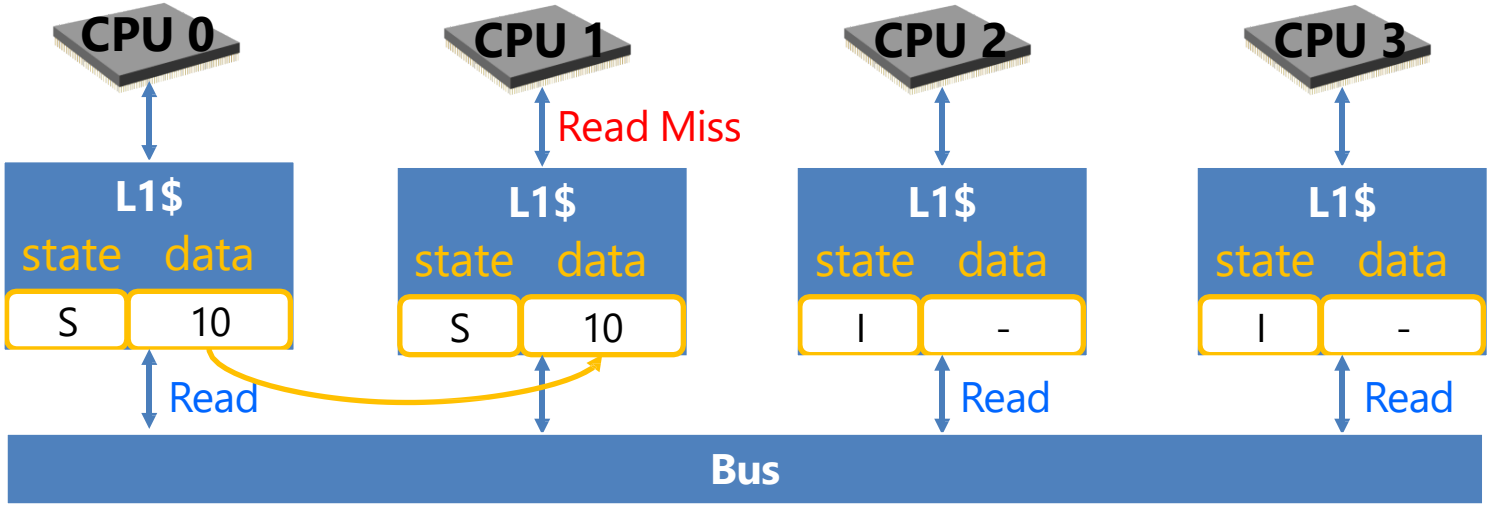
1. CPU 0: line in Modified state, CPU 1~3: line in Invalid state
2. CPU 1 reads line, causing a Read Miss

Scenario 1: Read Snooped on Bus



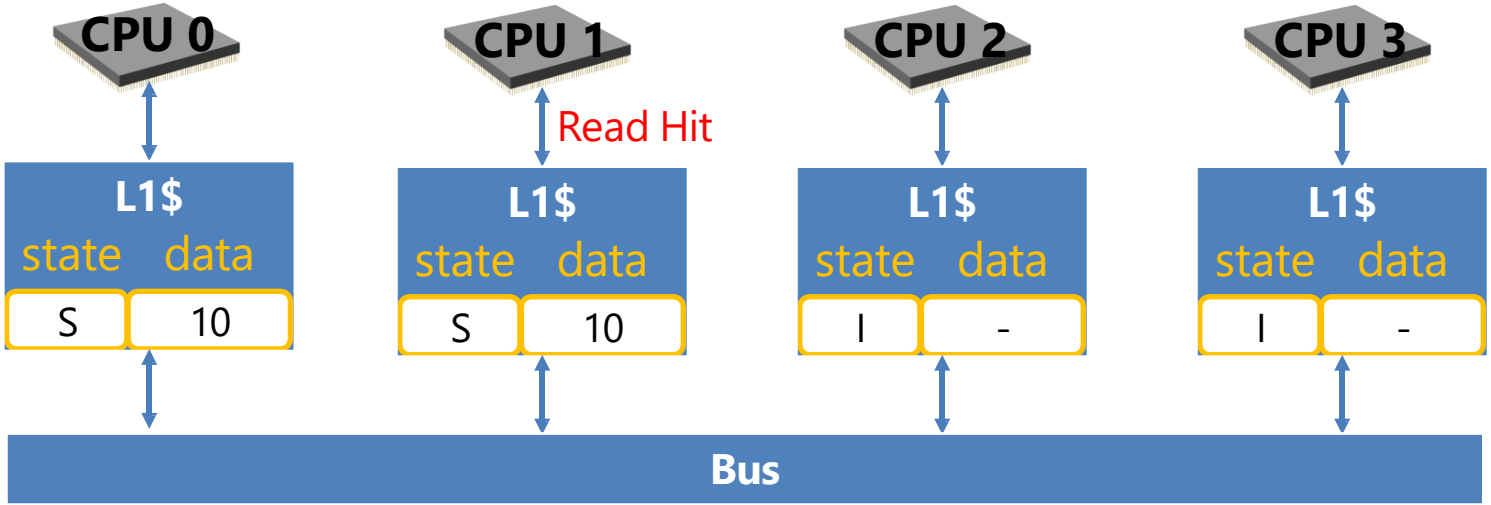
1. CPU 0: line in Modified state, CPU 1~3: line in Invalid state
2. CPU 1 reads line, causing a Read Miss
3. A Read message is broadcast on the Bus

Scenario 1: Read Snooped on Bus



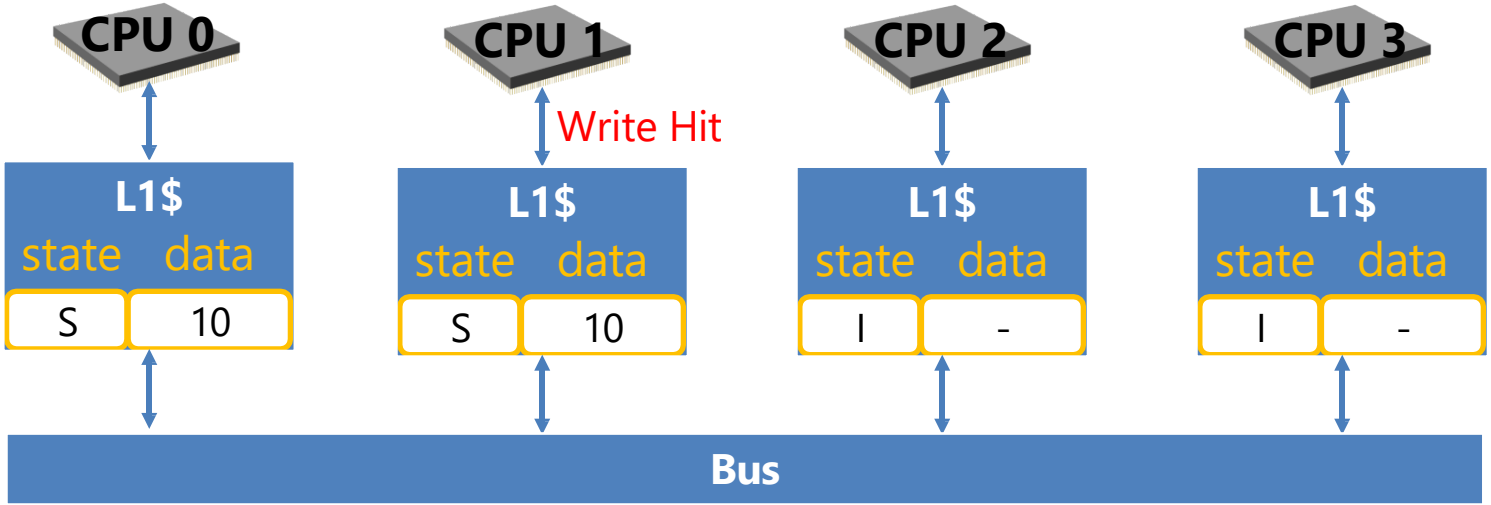
1. CPU 0: line in Modified state, CPU 1~3: line in Invalid state
2. CPU 1 reads line, causing a Read Miss
3. A Read message is broadcast on the Bus
4. CPU 0 snoops message and provides line to CPU1
   - Line in CPU 0 and CPU 1 both transition to Shared state

Scenario 1: Read Snooped on Bus



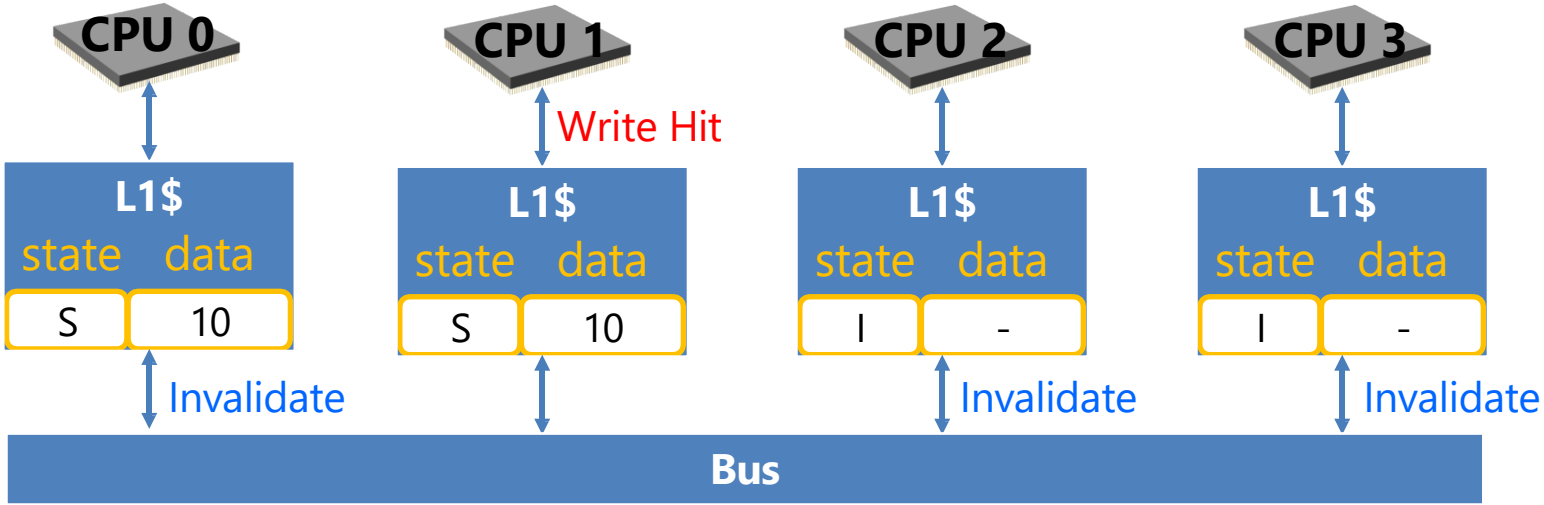
- Subsequent read hits on CPU 0 or CPU 1 don't generate messages
- Only read misses generate messages, not read hits
  → Reduces bus bandwidth pressure since misses are rare!

Scenario 2: Invalidate Snooped on Bus



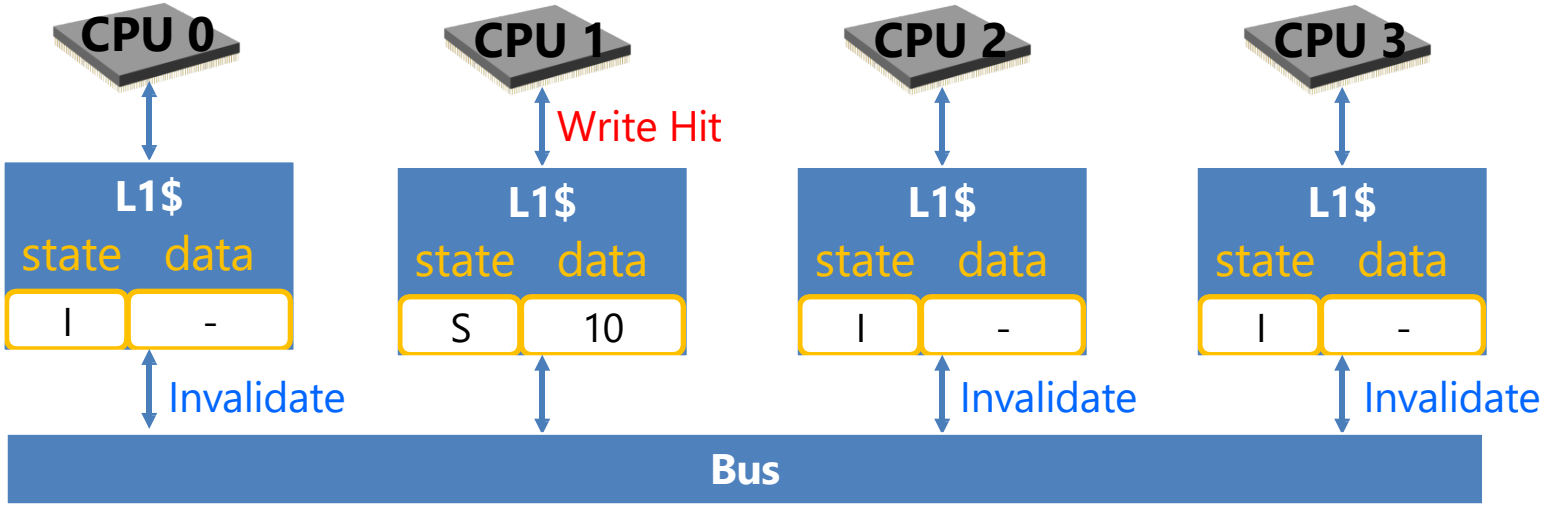
1. CPU 1 writes to line, causing a Write Hit

Scenario 2: Invalidate Snooped on Bus



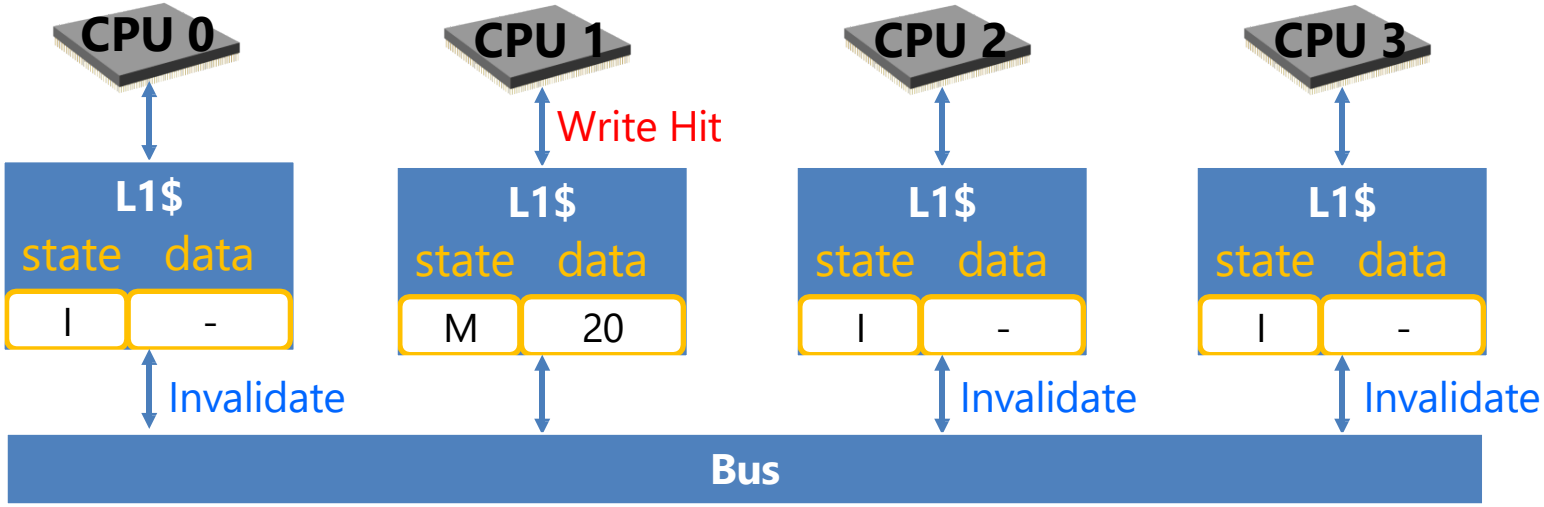
1. CPU 1 writes to line, causing a Write Hit
2. An Invalidate message is broadcast on the Bus
   - To remove all copies of the line that may become inconsistent

Scenario 2: Invalidate Snooped on Bus



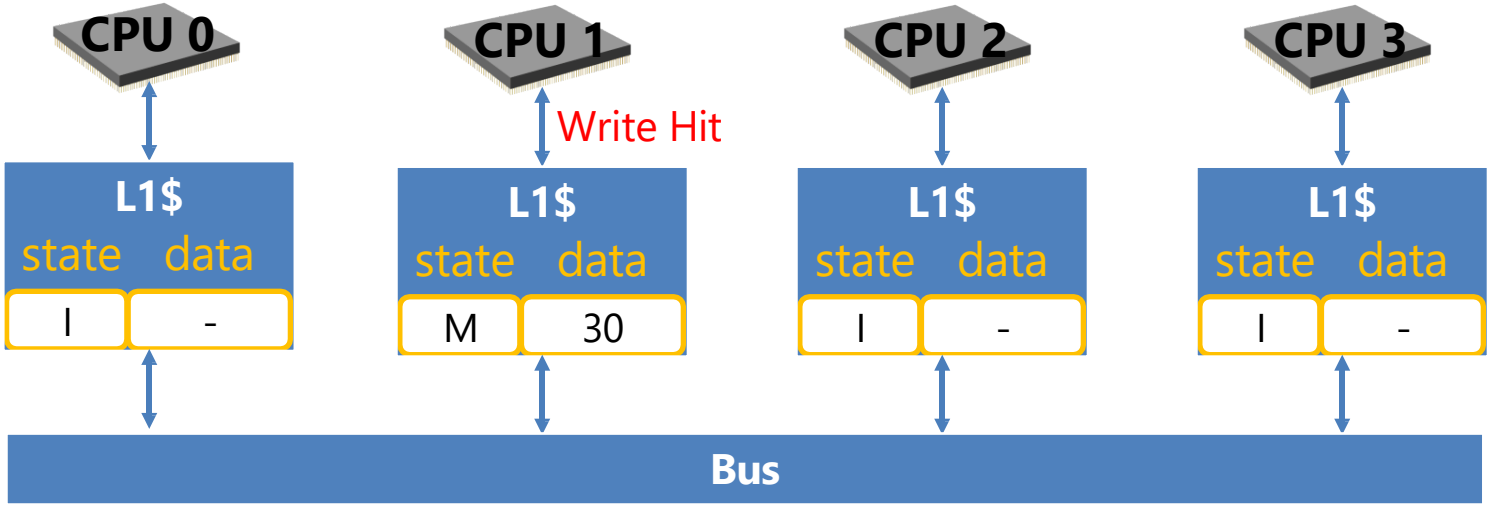
1. CPU 1 writes to line, causing a Write Hit
2. An Invalidate message is broadcast on the Bus
   - To remove all copies of the line that may become inconsistent
3. CPU 0 snoops message and invalidates its line

Scenario 2: Invalidate Snooped on Bus



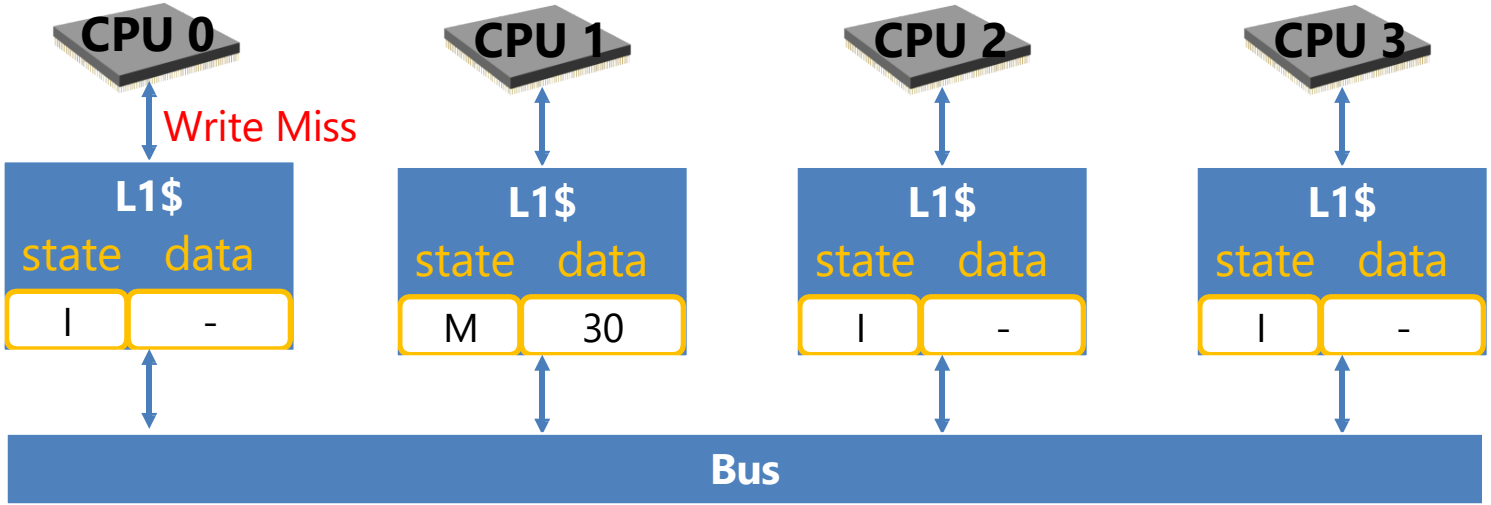
1. CPU 1 writes to line, causing a Write Hit
2. An Invalidate message is broadcast on the Bus
   - To remove all copies of the line that may become inconsistent
3. CPU 0 snoops message and invalidates its line
4. Line in CPU 1 transitions to Modified state

Scenario 2: Invalidate Snooped on Bus



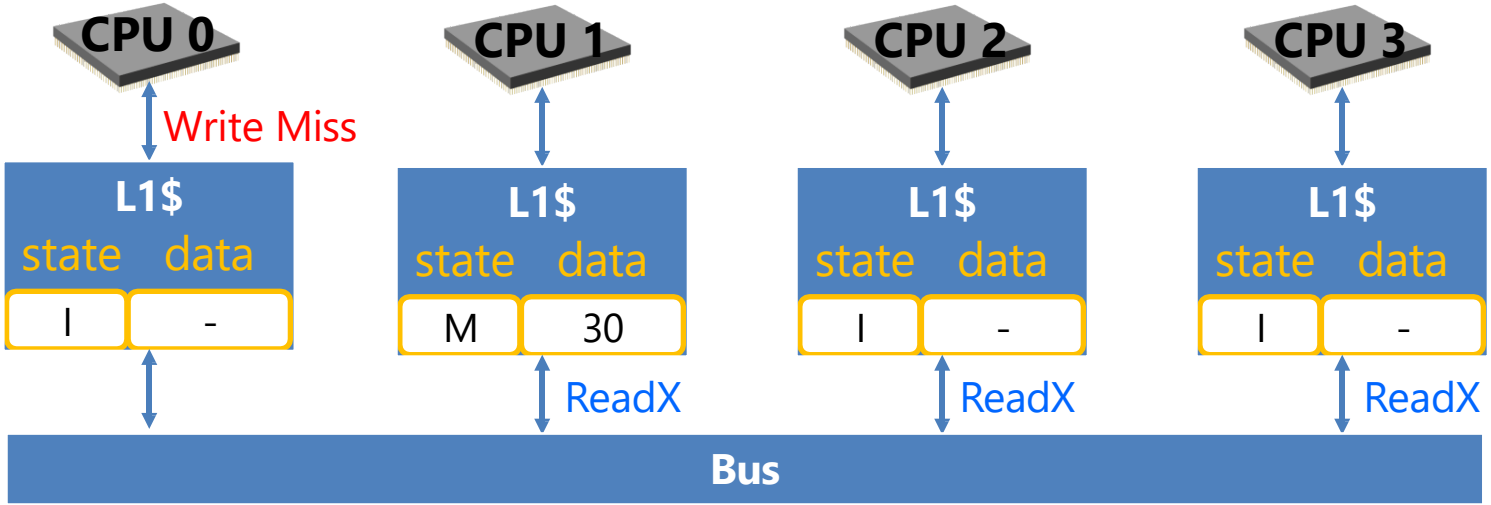
- Subsequent write hits on CPU 1 don't generate messages
- Only write hit in Shared state generate messages
  → Reduces bandwidth since most write hits are to Modified state

Scenario 3: Read Exclusive Snooped on Bus



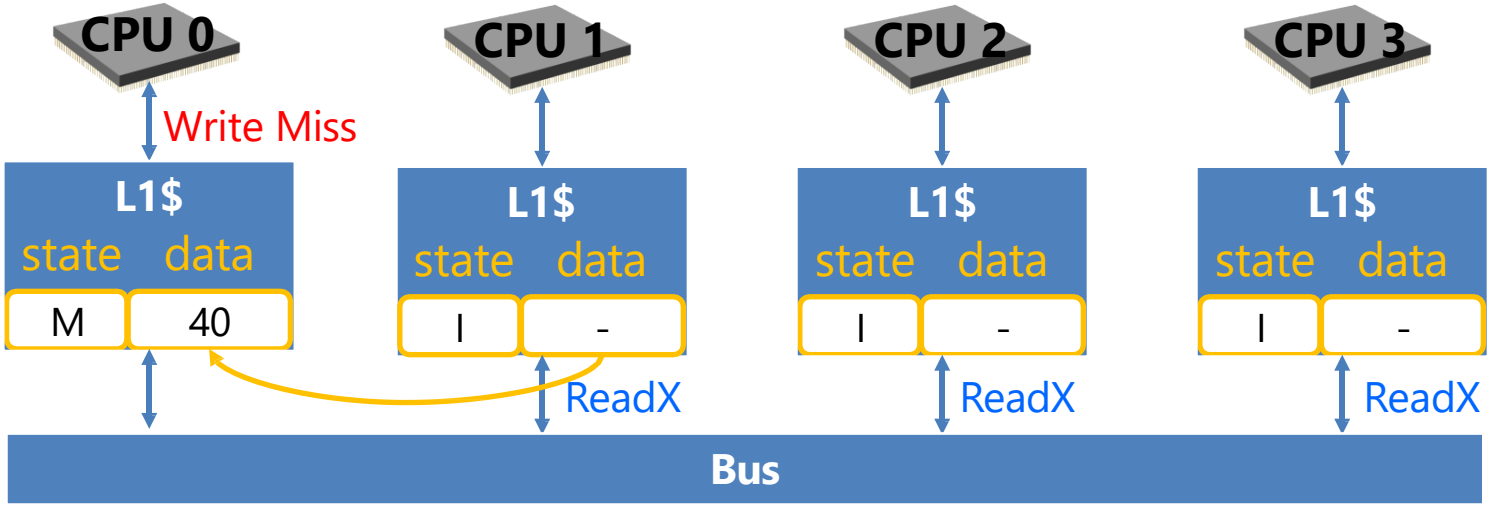
1. CPU 0 writes to line, causing a Write Miss

Scenario 3: Read Exclusive Snooped on Bus



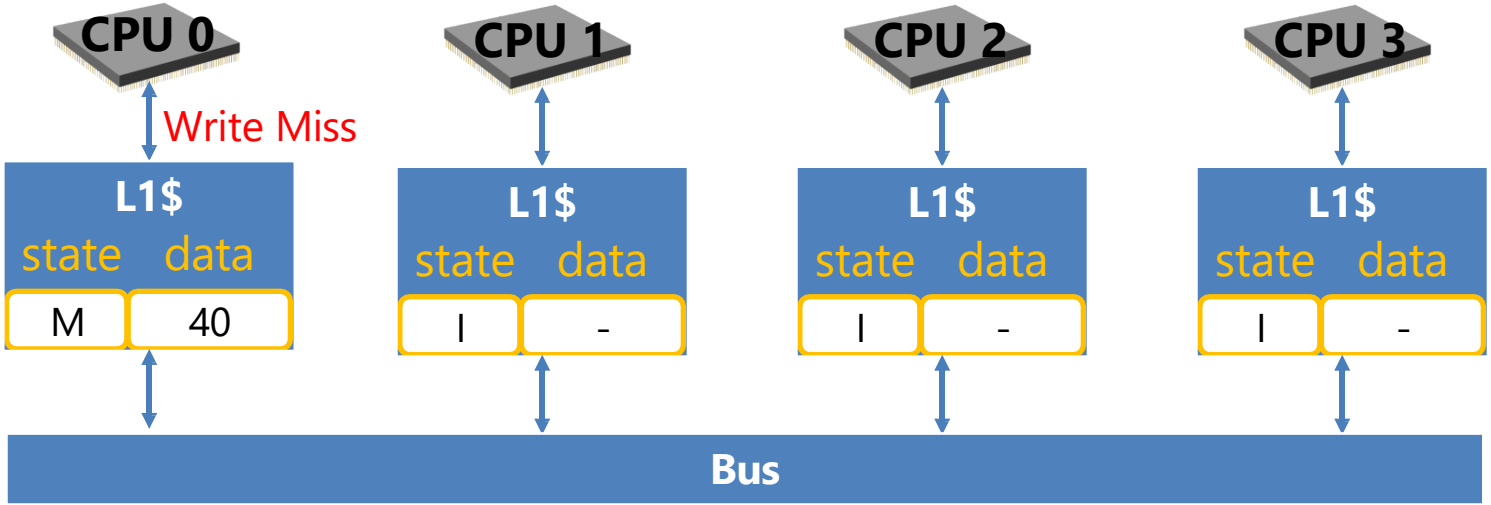
1. CPU 0 writes to line, causing a Write Miss
2. A ReadX (read exclusive) message is broadcast on the Bus
   - To read line into CPU 0 before writing to it (remember?)
   - To remove all copies of the line that may become inconsistent

Scenario 3: Read Exclusive Snooped on Bus



1. CPU 0 writes to line, causing a Write Miss
2. A ReadX (read exclusive) message is broadcast on the Bus
   - To read line into CPU 0 before writing to it (remember?)
   - To remove all copies of the line that may become inconsistent
3. CPU 1 snoops message and provides line to CPU 0
   - Line in CPU 1 is invalidated before line in CPU 0 is modified

Scenario 3: Read Exclusive Snooped on Bus



- Subsequent write hits on CPU 0 don't generate messages
- Write miss generates message, but not subsequent write hits
  → Reduces bus bandwidth pressure since misses are rare!

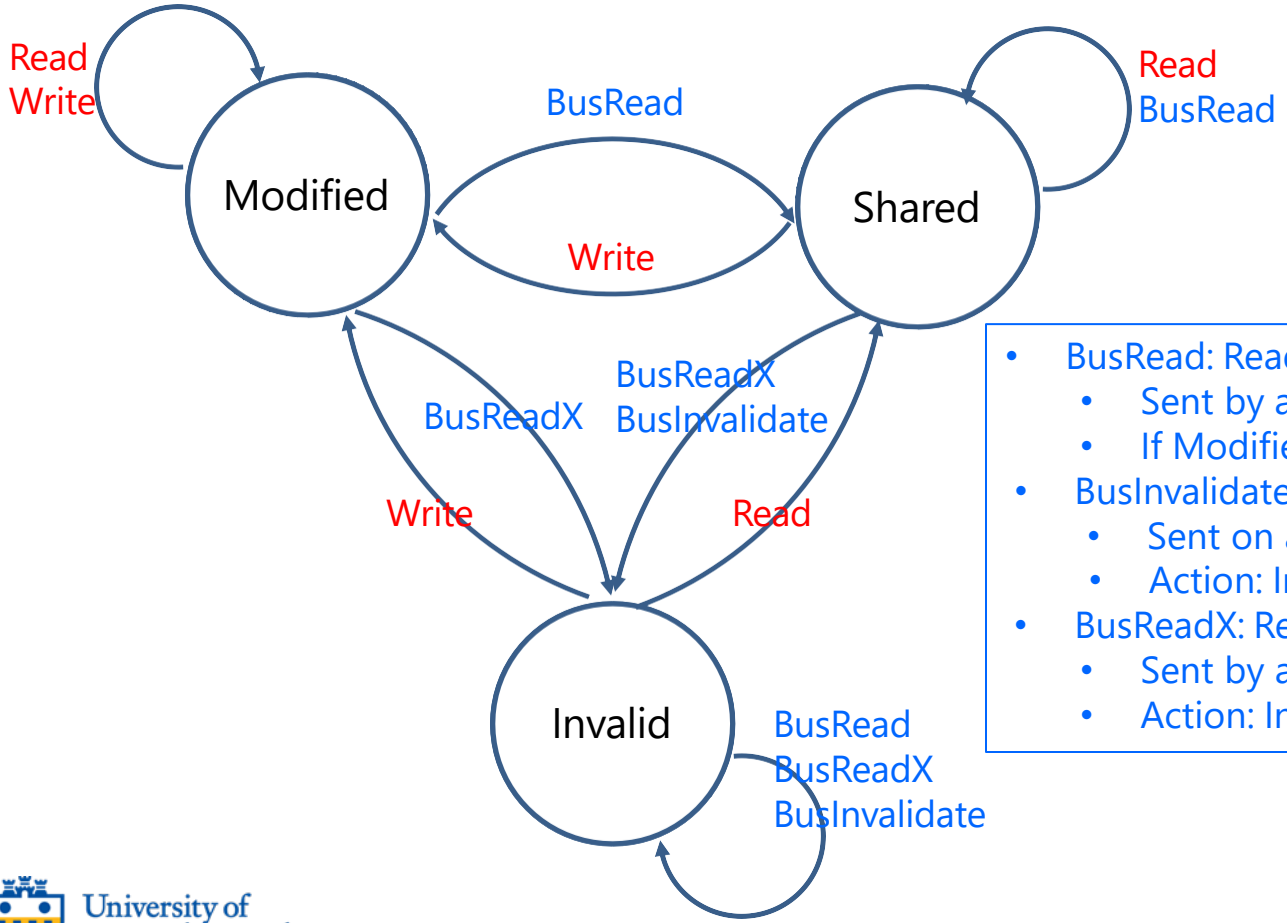
MSI: Example

- Invariant: a processor can modify data only when cache block is in M state
- Invalidate and ReadX bus messages help maintain that invariant

Event	P1's cache	P2's cache
	L (invalid)	L (invalid)
P1 writes 10 to A (write miss)	L ← A = 10 (modified)	Read Exclusive A (from write in P1) L (invalid)
P1 reads A (read hit)	L ← A = 10 (modified)	L (invalid)
P2 reads A (read miss)	Read A (from read in P2) L ← A = 10 (shared)	L ← A = 10 (shared)
P2 writes 20 to A (write hit)	Invalidate A (from write in P2) L (invalid)	L ← A = 20 (modified)
P2 writes 40 to A (write hit)	L (invalid)	L ← A = 30 (modified)
P1 write 50 to A (write miss)	L ← A = 40 (modified)	Read Exclusive A (from write in P1) L (invalid)

Cache Controller FSM for MSI Protocol

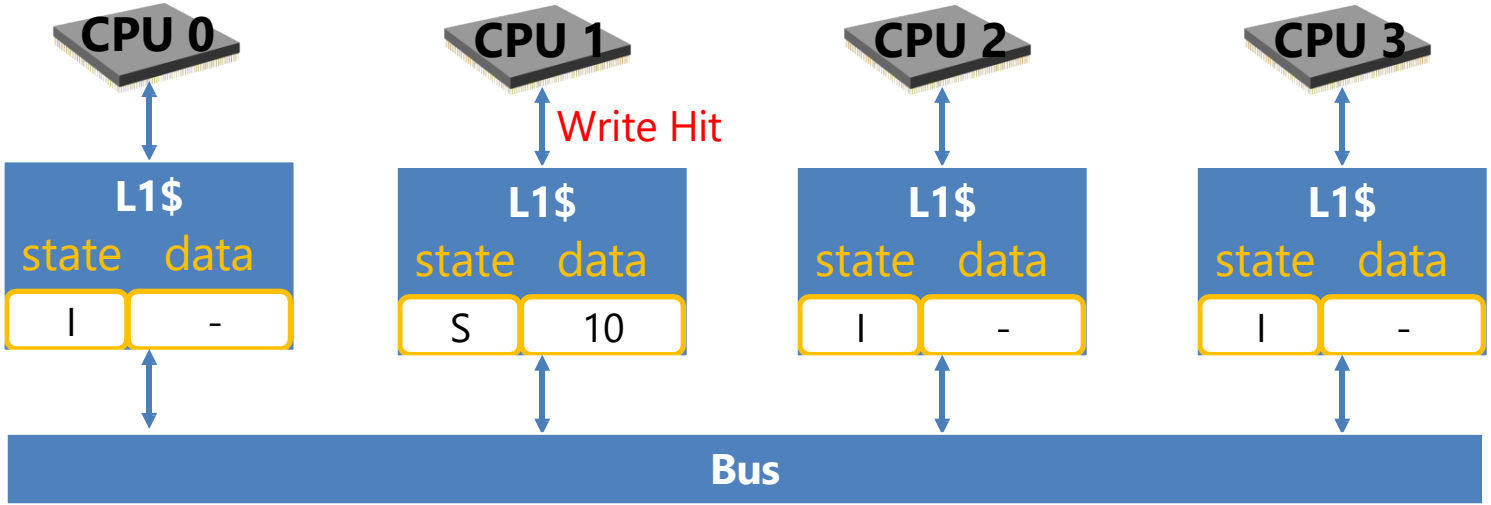
- Processor activity in red, Bus activity in blue



- BusRead: Read request is snooped
  - Sent by another processor on a read miss
  - If Modified, transition to Shared state
- BusInvalidate: Invalidate request is snooped
  - Sent on a write hit on shared cache line
  - Action: Invalidate shared line (if it exists)
- BusReadX: Read exclusive request is snooped
  - Sent by another processor on a write miss
  - Action: Invalidate shared line (if it exists)

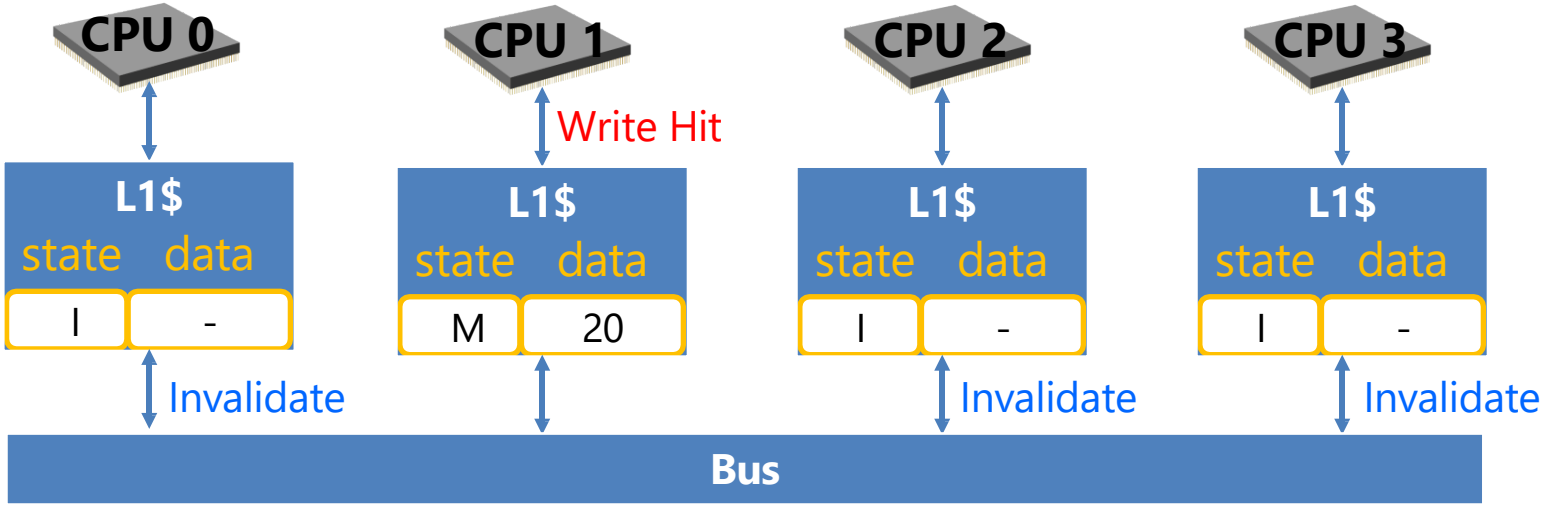
MESI Cache Coherence Protocol

MSI protocol sometimes sends redundant invalidates



1. CPU 1 reads line, causing a Read Miss

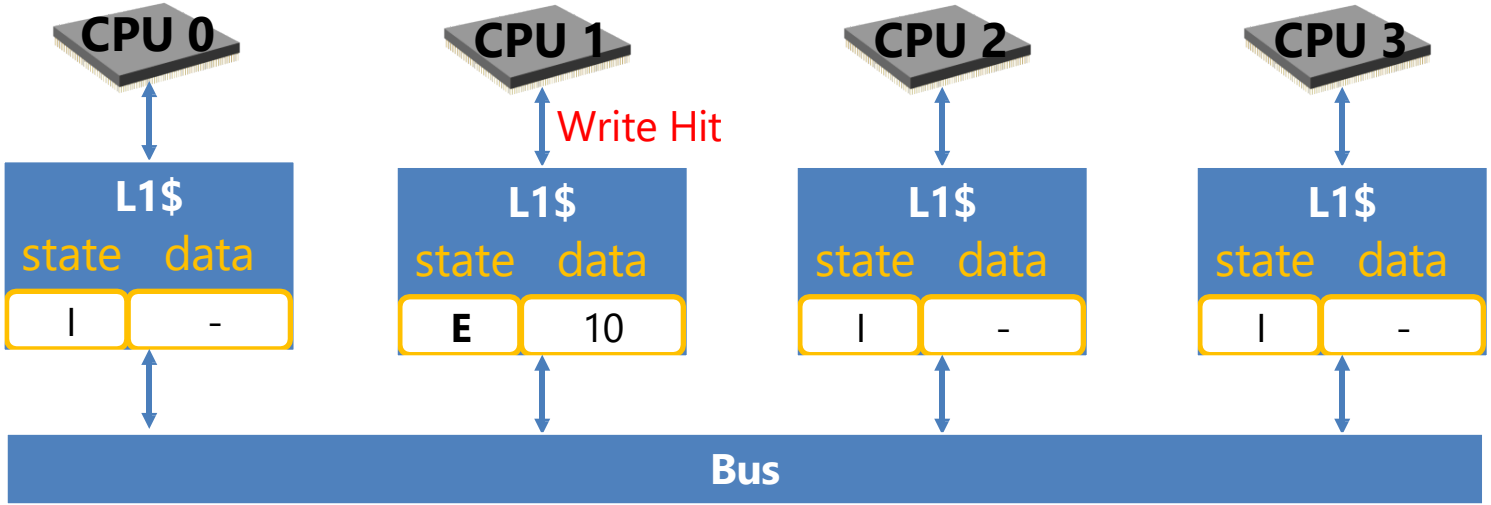
MSI protocol sometimes sends redundant invalidates



1. CPU 1 reads line, causing a Read Miss
2. CPU 1 immediately writes to line, causing a Write Hit
3. An Invalidate message is broadcast on the Bus
   - CPU 1 has no way of knowing it is the only copy because it is in S state

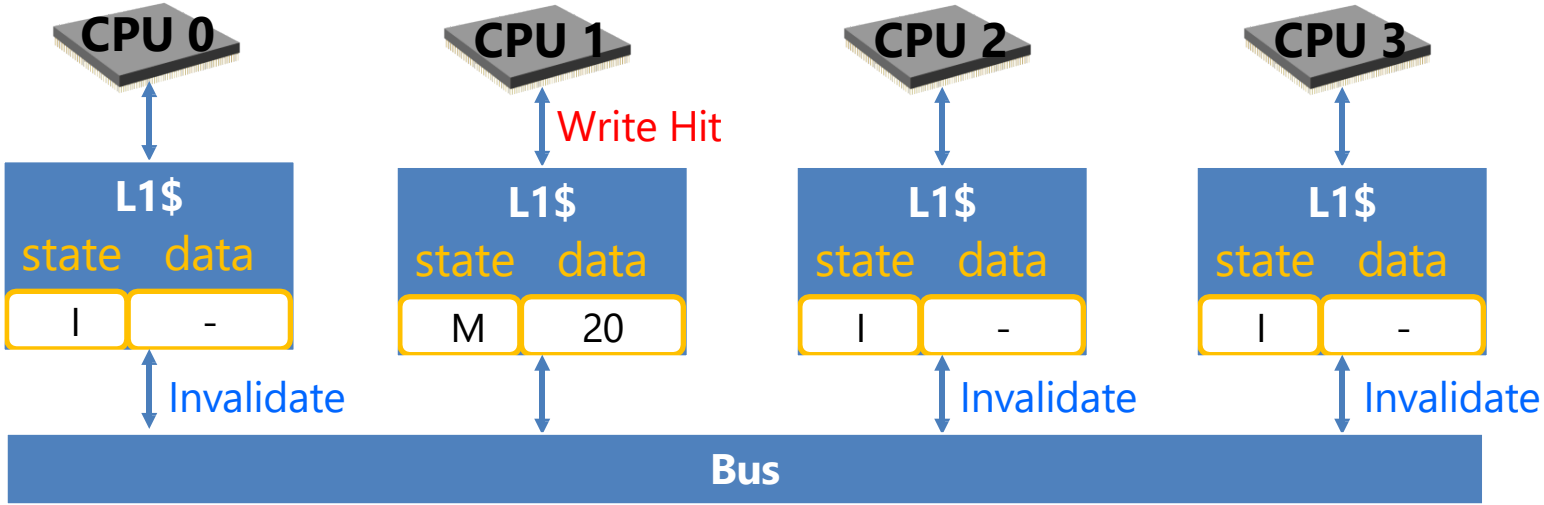
⇨ Source of redundant invalidates that saturates the bus bandwidth

MESI protocol avoids redundant invalidates



1. CPU 1 reads line, causing a Read Miss
   - Now line transitions to E (Exclusive) state instead of S (Shared) state

MSI protocol sometimes sends redundant invalidates

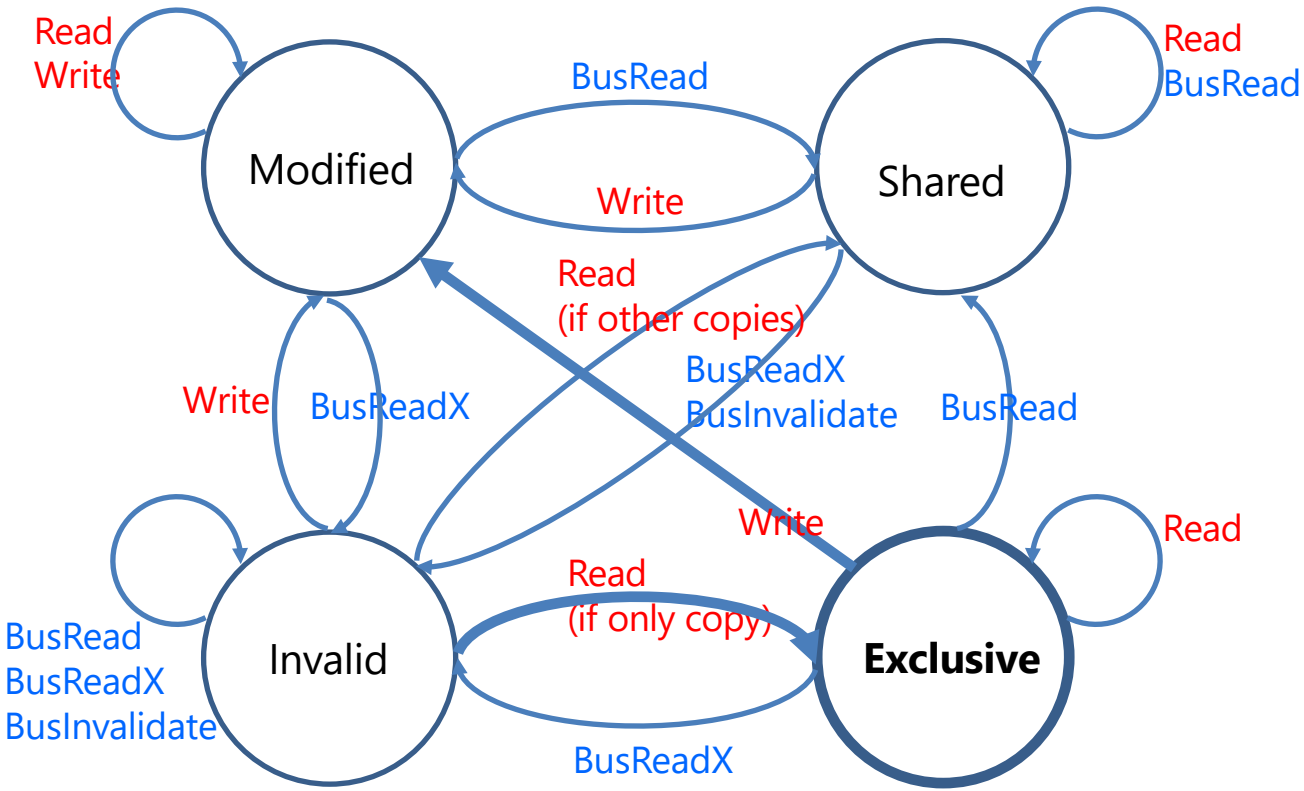


1. CPU 1 reads line, causing a Read Miss
   - Now line transitions to E (Exclusive) state instead of S (Shared) state
2. CPU 1 writes to line, but silently transitions to M state
   - Does not send invalidates since it has exclusive ownership of line

⇨ MESI protocol makes more efficient use of bus bandwidth

Cache Controller FSM for MESI Protocol

- Now an exclusive Read followed by a Write does not incur any bus activity



TLB Coherence

How about TLBs?

- We said TLBs are also a type of cache that caches PTEs.

- TLBs need to be kept coherent when page mapping changes
  - Specifically when page is unmapped or page protection changes
  - Or, program will have illegal access to that physical page
  - Not only TLB of local CPU but TLBs of all CPUs in the system

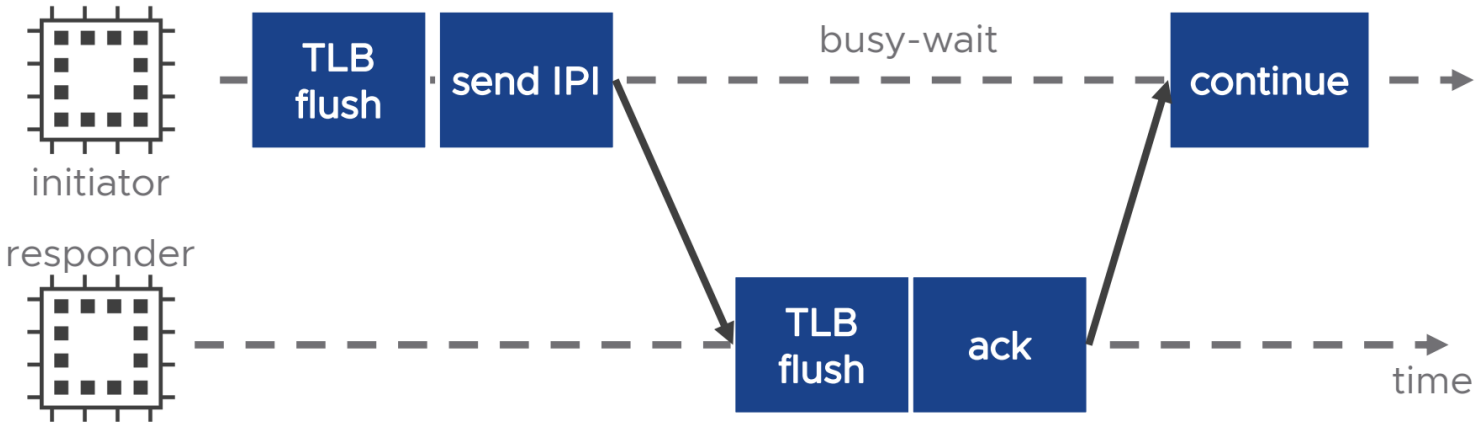
- Unfortunately, there is no hardware coherence for TLBs. ☹
  - Unlike caches, HW doesn't know when TLB entries need updating

- That means software (the OS) must handle the coherence
  - Which is of course much much slower

TLB shootdown

- In order to update a PTE (page table entry)
  - Initiator OS must first flush its own TLB
  - Send IPIs (Inter-processor interrupts) to other processors
    - To flush the TLBs for all other processors too
  - Source of significant performance overhead

TLB Flushes in Linux and FreeBSD



* Courtesy of Nadav Amit et al. at VMWare